

Documentation

Object Tracking using YOLOv10 and Deep-SORT

1. Project Overview

This project implements a real-time **person detection and multi-object tracking system** on a video file.

The system detects people in each frame using a deep learning model and assigns a unique ID to each detected person to track them across frames.

The main goal is to combine object detection and tracking to maintain object identity throughout the video sequence.

2. Technologies and Tools

- **Programming Language:** Python
- **Computer Vision Library:** OpenCV
- **Object Detection Model:** YOLOv10 (Ultralytics implementation)
- **Tracking Algorithm:** Deep-SORT (Deep Simple Online and Realtime Tracking)

Model Used

- YOLOv10 medium version (yolov10m)

3. System Architecture

The system is modular and consists of three main components:

3.1 Main Module

Responsible for:

- Loading the video file
- Initializing the detection and tracking modules

- Reading frames sequentially
 - Performing detection and tracking
 - Displaying results
 - Measuring system performance (FPS)
-

3.2 Detection Module (YOLO Detector)

This module:

- Loads a pre-trained YOLOv10 model
- Performs object detection on each frame
- Filters detections to keep only the "person" class
- Extracts bounding box coordinates and confidence scores
- Outputs formatted detections for the tracker

The detection process ensures that only relevant objects (persons) are passed to the tracking system.

3.3 Tracking Module (Deep-SORT)

The tracking module:

- Receives detections from the detector
- Assigns a unique tracking ID to each person
- Maintains identity across frames
- Handles temporary occlusions
- Removes tracks when objects disappear

Deep-SORT uses:

- Motion information
- Appearance embedding (feature extraction)
- Data association techniques

to ensure stable multi-object tracking.

4. Processing Workflow

For each video frame, the following steps are performed:

1. Read frame from video
2. Perform object detection using YOLOv10
3. Filter only person detections
4. Send detections to Deep-SORT tracker
5. Update tracks and assign unique IDs
6. Draw bounding boxes and tracking IDs
7. Calculate and print FPS
8. Display processed frame

This pipeline repeats until the video ends or the user exits.

5. Input and Output

Input

- A video file containing multiple persons (e.g., football match video)

Output

- Real-time video display with:
 - Bounding boxes around detected persons
 - Unique tracking ID for each person
 - FPS (Frames Per Second) printed in the console
-

6. Performance Considerations

System performance depends on:

- Hardware specifications (CPU/GPU)
- Model size (YOLOv10m)
- Video resolution
- Number of detected objects

The FPS is calculated dynamically for each frame to measure real-time performance.

7. Advantages of the System

- Real-time processing capability

- Accurate person detection
 - Stable ID assignment across frames
 - Modular and extendable design
-

8. Possible Extensions

The system can be extended for:

- Surveillance systems
- Crowd monitoring
- Sports analytics
- Traffic monitoring
- Behavior analysis

Additional improvements may include:

- Counting people
 - Heatmap generation
 - Performance optimization
 - GPU acceleration
-

9. Conclusion

This project successfully integrates deep learning-based object detection (YOLOv10) with a robust multi-object tracking algorithm (Deep-SORT).

The combination of detection and tracking enables persistent identity assignment and real-time analysis of moving individuals in video sequences.
